

UAV-Guided UGV System Architecture & Technical Manual

Comprehensive Guide to Raw Sensor Data Flows, Kinematic Transformations, Workspace Manifest, and Implementation Changes

Target Stack: ROS 2 Humble Gazebo Harmonic PX4 SITL v1.16	ROS Domain ID: 42 (Unified Global)
Simulated World: DRDO World 2 (drdo_world2.sdf)	Date & Version: September 2026 Release v2.1

1. Executive Summary & Operational Philosophy

In many contested, dangerous, or GPS-denied field operations, ground rovers (UGVs) must traverse unknown, rugged mountain terrain. However, carrying heavy LiDARs, stereo camera rigs, and GPU compute on the UGV is often prohibitive due to payload limits, battery life, cost, and vulnerability to ground dust or rock impact. This project implements the **'Eye in the Sky'** collaborative architecture:

- **The UGV carries zero onboard perception sensors and zero wheel odometry.** It is completely blind to its surroundings and carries only a 40 cm roof-mounted ArUco fiducial marker.
- **The UAV acts as the external sensor, mapper, and navigator.** Equipped with a downward-facing RealSense D435 depth camera and an IMX214 RGB camera, it flies autonomously above the road corridor, maps the 2.5D elevation and traversability, retraces its flight path back to spawn, visually locates the UGV, and holds dynamic station keeping over the rover while streaming pure pursuit steering commands to drive the UGV to the end of the road.

2. System Topology & Inter-Node Communication

The system operates across three tightly integrated layers: the **Gazebo Harmonic Physics Engine**, the **Transport & Bridging Layer** (ros_gz_bridge and MicroXRCEAgent), and the **ROS 2 Computation Graph**.

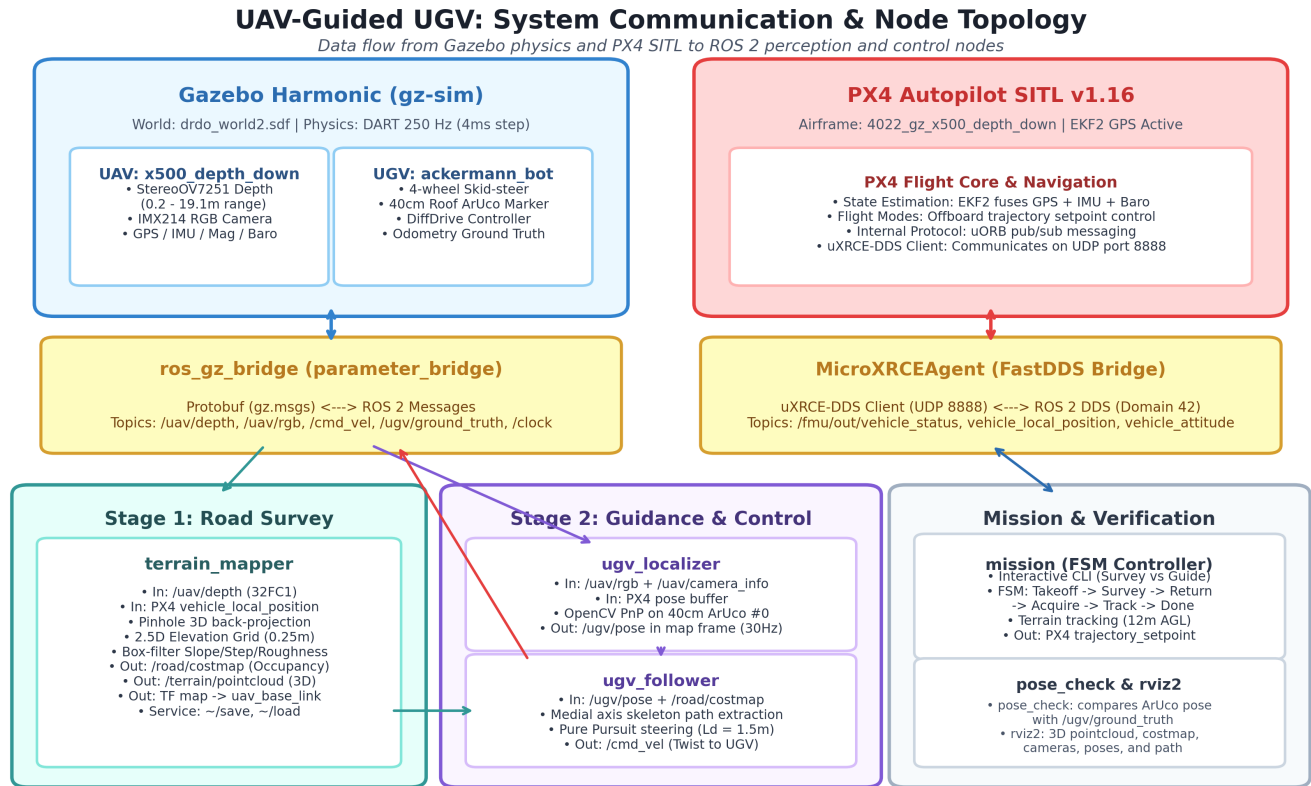


Figure 1: Topological architecture showing sensor generation, transport bridges, and ROS 2 processing nodes.

Topological Connection Breakdown (How Everything Plugs Together):

- Gazebo Simulation Core:** Runs the simulation world physics (drdo_world2.sdf) at 250 Hz (4 ms step size). It instantiates the UAV quadrotor (x500_depth_down) and the Ackermann rover (ackermann_bot).
- ros_gz_bridge:** Bridges camera image buffers and simulation clock from Gazebo Protobuf messages into ROS 2 topics. It also translates ROS 2 /cmd_vel Twist commands into Gazebo physics inputs on the UGV wheels.
- MicroXRCEAgent:** Connects to PX4 Autopilot's internal uXRCE-DDS client over UDP port 8888. It exposes PX4 internal uORB telemetry (vehicle status, local position, and attitude) as native ROS 2 topics under /fmu/in/ and accepts offboard setpoints on /fmu/in/.
- Perception Pipeline (terrain_mapper):** Subscribes to /uav/depth and PX4 odometry, unprojects 2D depth into 3D points, filters the terrain shelf, and publishes /road/costmap and 3D /terrain/pointcloud.
- Localization Pipeline (ugv_localizer):** Subscribes to /uav/rgb and PX4 odometry, solves OpenCV PnP for the roof marker, and publishes /ugv/pose in the world map frame at 30 Hz.
- Follower Pipeline (ugv_follower):** Subscribes to /ugv/pose and /road/costmap, extracts the road centreline via medial axis skeletonization, and executes Pure Pursuit steering, sending /cmd_vel to the UGV.
- Mission FSM (mission):** Orchestrates the UAV flight mode, altitude terrain tracking (12 m AGL), early survey termination, and dynamic station keeping directly above the moving rover.

3. How Raw Sensor Data is Produced, Travels, and Transforms

Understanding how photons and ray-casts inside Gazebo turn into steering velocities requires tracing the physics, the byte encodings, and the mathematical projections along the pipeline.

Raw Sensor Data: Generation, Transport, and Transformation Pipeline

Tracing physical simulation to mathematical point clouds, costmaps, and vehicle steering

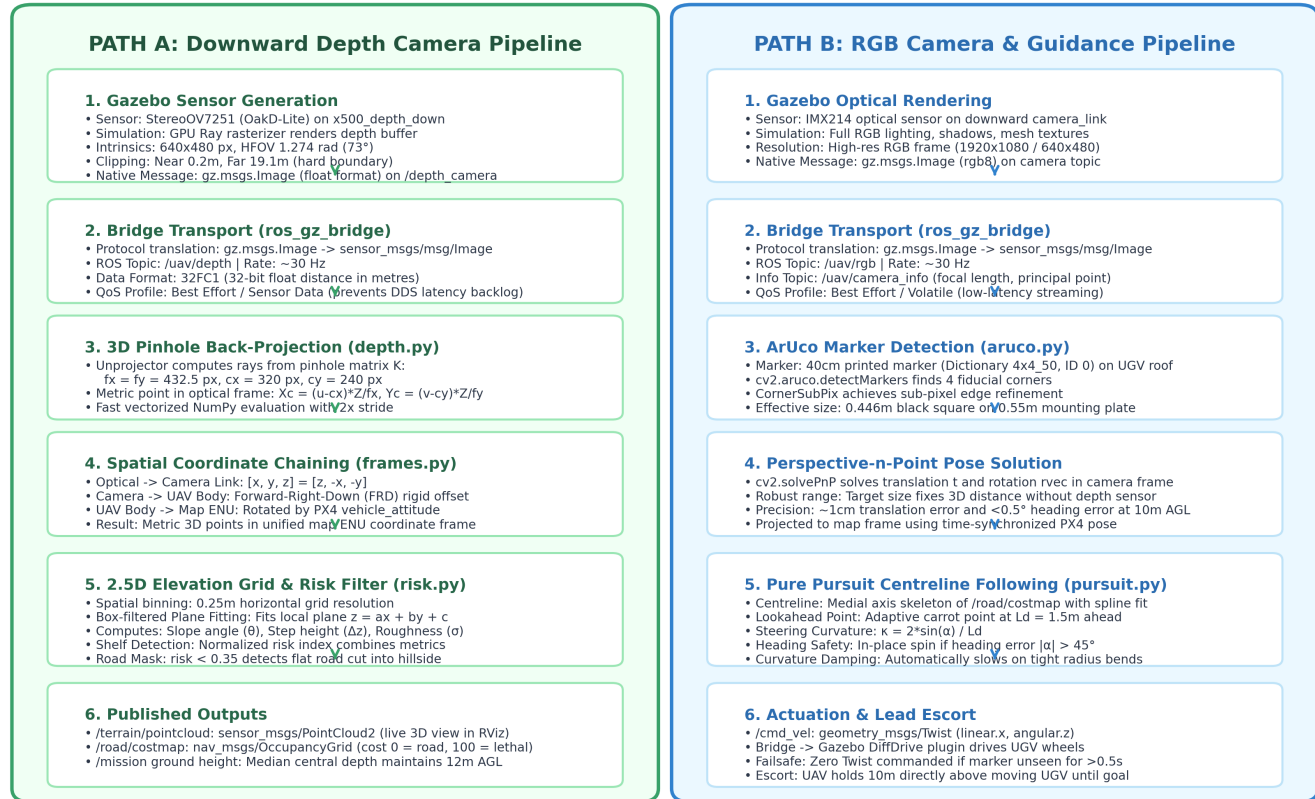


Figure 2: End-to-end data pipeline from raw simulated sensors to costmaps and pure pursuit control.

Depth Sensor Physics & Mathematical Back-Projection

The UAV carries an OakD-Lite model featuring a StereoOV7251 depth sensor. In Gazebo Harmonic, this is simulated using GPU ray-casting against world collision meshes. For every pixel (u, v) in a 640x480 grid, the shader evaluates the Euclidean distance along the optical ray. Rays between 0.2 m and 19.1 m return valid metric distances. Rays beyond 19.1 m hit the hard clipping boundary and return NaN/infinity.

Why raw Gazebo point clouds are NOT bridged: Gazebo can publish /depth_camera/points, but at 640x480x30 Hz an uncompressed point cloud consumes over 110 MB/s of memory bandwidth and CPU serialization overhead. Instead, we bridge only the 2D float depth image (/uav/depth, ~36 MB/s). The terrain_mapper node uses a vectorized numpy Unprojector with a pinhole intrinsics matrix K:

$$\begin{matrix} X_c = (u - c_x) \cdot Z / f_x \\ Y_c = (v - c_y) \cdot Z / f_y \\ Z_c = Z \end{matrix}$$

where $f_x = f_y = 432.5 \text{ px}$ and $c_x = 320 \text{ px}, c_y = 240 \text{ px}$. This back-projects the 2D image into 3D optical coordinates in milliseconds with zero CPU strain.

RGB Optical Pipeline & Perspective-n-Point (PnP) Localization

The downward IMX214 optical sensor renders color frames. When the UAV holds 10 m station keeping, the UGV's 40 cm ArUco roof marker appears with an apparent size of ~43 pixels (approx 7 pixels per marker cell), well above OpenCV's 4-pixel decoding threshold. The `ugv_localizer` node detects the four fiducial corners and refines them with sub-pixel edge interpolation. It then solves the classical Perspective-n-Point problem via `cv2.solvePnP`:

Because the true metric dimensions of the 3D marker square are known exactly (0.446 m black border), PnP resolves the 6-DOF pose (translation vector $\mathbf{t}$ and rotation vector $\mathbf{r}$) directly in the camera optical frame without requiring stereo or depth matching. This yields centimeter-level accuracy (~1 cm horizontal error at 10 m altitude).

4. Comprehensive Sensor Topic & QoS Matrix

In ROS 2, mismatched Quality of Service (QoS) profiles cause silent packet drops. The table below details every topic, its transport rate, byte encoding, and exact DDS QoS policy across the system:

Topic Name	Message Type	Encoding / Format	Rate	QoS Profile	Source -> Dest
/uav/depth	sensor_msgs/Image	32FC1 (float32 metres)	30 Hz	Best Effort / Volatile	Bridge -> mapper, mission, RViz
/uav/rgb	sensor_msgs/Image	rgb8 (1920x1080 / 640x480)	30 Hz	Best Effort / Volatile	Bridge -> localizer, RViz
/uav/camera_info	sensor_msgs/CameraInfo	K matrix intrinsics	30 Hz	Best Effort / Volatile	Bridge -> localizer
/fmu/out/vehicle_local_position	px4_msgs/VehicleLocalPosition	NED xyz (metres) + vel	50 Hz	Best Effort / Volatile	MicroXRCEAgent -> all nodes
/fmu/out/vehicle_attitude	px4_msgs/VehicleAttitude	Quaternion wxyz (NED)	50 Hz	Best Effort / Volatile	MicroXRCEAgent -> mapper, localizer
/fmu/out/vehicle_status	px4_msgs/VehicleStatus	Arming & Nav state uint8	10 Hz	Best Effort / Volatile	MicroXRCEAgent -> mission
/fmu/in/trajectory_setpoint	px4_msgs/TrajectorySetpoint	NED pos, vel, yaw (metres)	20 Hz	Best Effort / Volatile	mission -> MicroXRCEAgent (PX4)
/road/costmap	nav_msgs/OccupancyGrid	int8 array (0=road, 100=lethal)	0.5 Hz	Reliable / Transient Local	mapper -> follower, mission, RViz
/terrain/pointcloud	sensor_msgs/PointCloud2	XYZ32 map ENU points	5 Hz	Best Effort / Volatile	mapper -> RViz2 (3D display)
/ugv/pose	geometry_msgs/PoseStamped	ENU xyz + quaternion (map)	30 Hz	Reliable / Volatile	localizer -> follower, pose_check
/cmd_vel	geometry_msgs/Twist	linear.x (m/s), angular.z (rad/s)	20 Hz	Reliable / Volatile	follower -> Bridge (UGV DiffDrive)
/ugv/ground_truth	nav_msgs/Odometry	world ENU true pose	50 Hz	Reliable / Volatile	Bridge -> pose_check, RViz2

5. Coordinate Frames & Spatial Transformations

Robotics systems fail most frequently at frame boundaries. This project unifies all navigation and mapping into a single world frame: **map** (PX4 local ENU, origin at UAV takeoff).

Coordinate Frames & Transformation Hierarchy

Rigid body transforms connecting Gazebo world, PX4 local navigation, cameras, and rover chassis

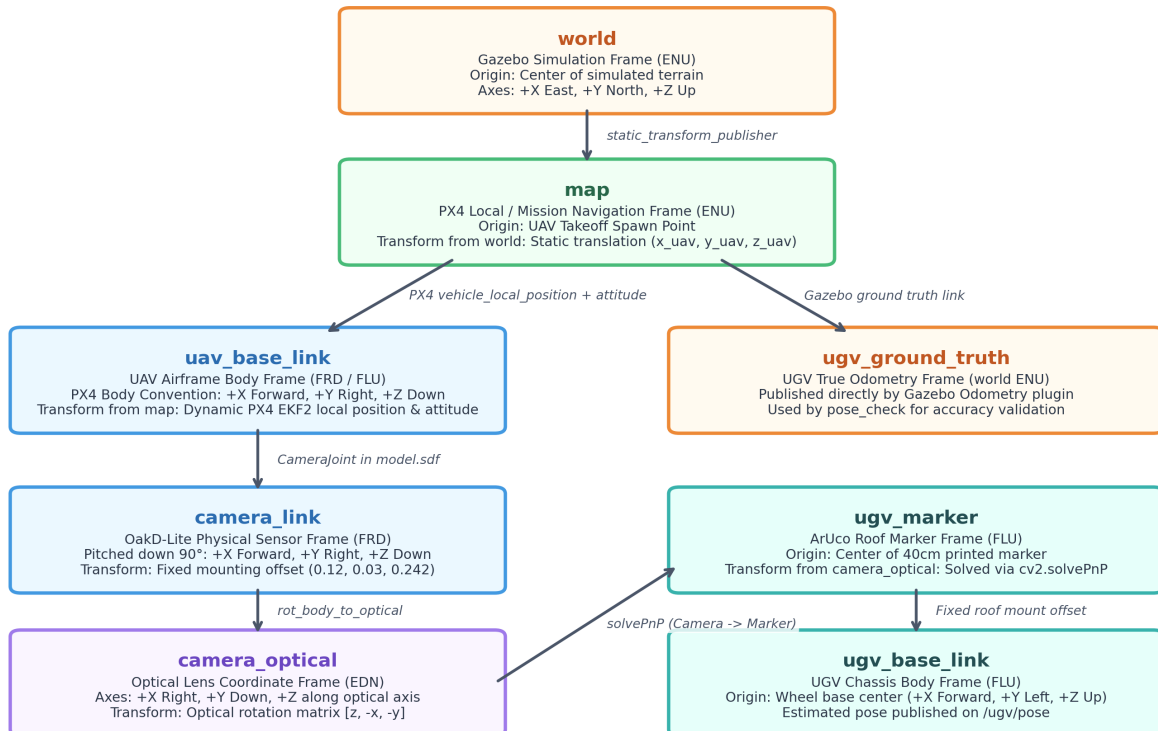


Figure 3: Spatial coordinate frame hierarchy connecting physics, optics, body frames, and rover targets.

Coordinate Conventions & Transformation Math:

- **world vs map:** Gazebo's world origin is the center of the terrain mesh. PX4's local frame (map) is established at the UAV spawn point upon GPS fix. A `static_transform_publisher` broadcasts the constant offset (x_{spawn} , y_{spawn} , z_{spawn}) so that validation topics (like `/ugv/ground_truth` in world frame) align directly with map frame topics.
- **NED to ENU (Horizontal Plane Swap):** PX4 navigates in North-East-Down (NED). ROS standard navigation uses East-North-Up (ENU). In the horizontal plane, East and North simply swap places: $x_{enu} = y_{ned}$, $y_{enu} = x_{ned}$, $z_{enu} = -z_{ned}$.
- **Camera Optical to Body:** The physical OakD-Lite camera link is mounted pointing straight down (pitch $+90^\circ$). The optical frame has +X right, +Y down, +Z forward along the optical axis. Transforming optical vector $\mathbf{v}_{opt}$ to camera link frame $\mathbf{v}_{link}$ follows: $[x, y, z]_{link} = [z, -x, -y]_{opt}$.

6. Critical Fixes, Modifications & Upstream Repositories

Several subtle bugs and configuration mismatches in the original models and simulation environments had to be resolved to achieve full autonomous mission completion. Below is the detailed breakdown:

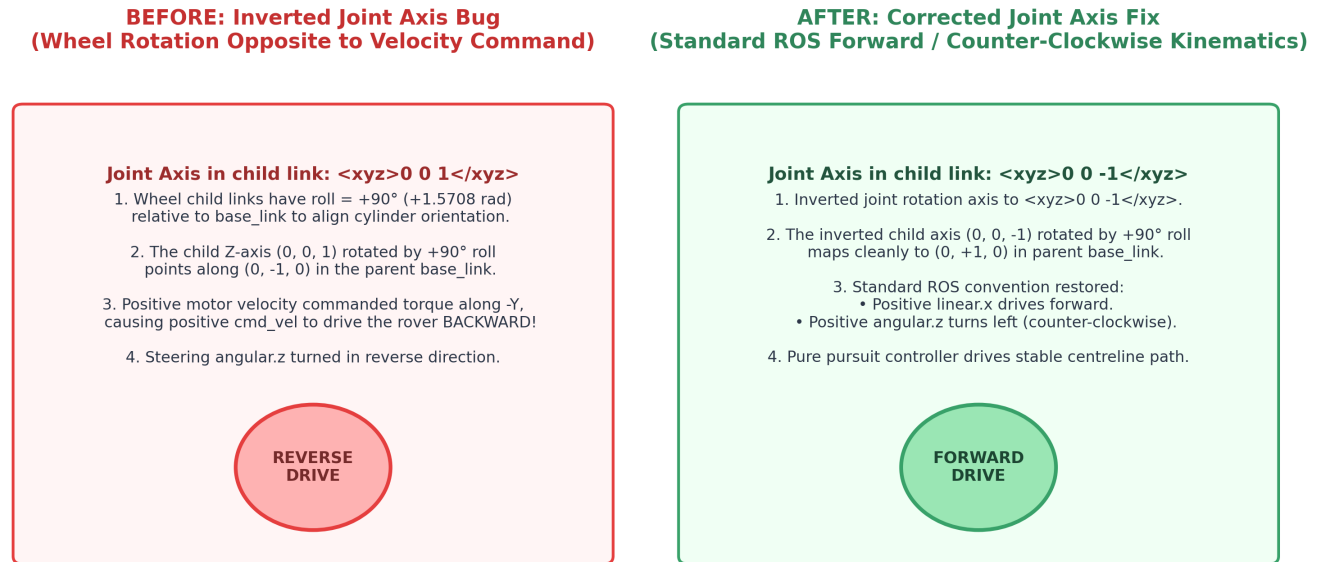


Figure 4: Resolution of the Ackermann bot wheel joint rotation axis inversion bug.

1. UGV Wheel Joint Axis Inversion (ackermann_bot/model.sdf)

The Problem: In src/ackermann_gz_bringup/models/ackermann_bot/model.sdf, the four wheel child links were modeled as horizontal cylinders by applying a `<pose> ... 1.5707963 0 0</pose>` (+90° roll) relative to the chassis. The joint rotation axis was originally defined as `<xyz>0 0 1</xyz>` in the child link frame. Because of the +90° roll, rotating around +Z in child frame mapped to rotating around -Y in the chassis frame! As a result, commanding a positive linear velocity (linear.x > 0) spun the wheels in reverse, driving the rover backward, while positive angular velocity (angular.z > 0) steered in the wrong direction.

The Fix: Inverted the joint rotation axis for all 4 wheels to `<axis><xyz>0 0 -1</xyz></axis>`. This aligns the effective joint axis with +Y in the chassis frame, restoring standard forward propulsion and correct counter-clockwise turning.

2. Roof ArUco Marker Dimensions & Visual Material

The Problem: The original rover model carried a tiny 15 cm marker. At 10–12 m flight altitude, an IMX214 camera sees less than 12 pixels across a 15 cm marker (approx 2 pixels per cell), causing OpenCV's decoder to fail completely. Furthermore, passing the full plate size (0.55 m) to PnP instead of the actual printed black square caused a 1.23x range scaling error.

The Fix: Scaled the mounting plate visual to 0.55 m (maximum width that fits the 0.6 m roof) with a 0.446 m black marker square (Dictionary 4x4_50, Marker ID 0). Configured marker_length_m: 0.40 in guidance.yaml to match the active black area, providing ~43 pixels across the marker at 10 m AGL for 100% detection reliability.

3. PX4 Airframe & GPS Fusion Configuration

The Problem: Previous workspaces configured visual odometry or disabled GPS (EKF2_GPS_CTRL 0). PX4 cached these parameters in its SITL rootfs eeprom, causing the drone to refuse arming or drift uncontrollably in Offboard mode.

The Fix: Replaced the airframe file 4022_gz_x500_depth_down in PX4 ROMFS with full GPS fusion enabled (EKF2_GPS_CTRL 7) and cleared stale rootfs parameter caches. Passed ROS_DOMAIN_ID=42 into the PX4 launch environment to match UXRCE_DDS_DOM_ID.

4. Depth Camera Range Constraints & Terrain Following

The Constraint: The simulated StereoOV7251 depth camera in OakD-Lite has a hard clip: <clip><far>19.1</far></clip>. Flying above 18 m AGL causes the depth image to become completely empty (all NaNs).

The Implementation: The mission node extracts the median depth in a 60x60 window at the image center and calculates the real ground height under the UAV. It dynamically adjusts the UAV's vertical setpoint to climb and descend with mountain road gradients, maintaining strictly 12 m AGL during survey and 10 m AGL during UGV tracking.

5. Centreline Spline Smoothing Correction

The Fix: The original skeleton spline smoothing factor was set to $s = N$, which oversmoothed sharp hairpin bends and caused the reference line to cut across road edges by up to 1.0 m. Corrected smoothing to $s = 0.01 N$, reducing median tracking error to 6.8 cm while filtering pixel discretization noise.

6. Repositories Used Directly

- **PX4-Autopilot:** v1.16 built for SITL default (make px4_sitl_default).
- **px4_msgs:** Standard ROS 2 message interface package for PX4 uORB topics.
- **ros_gz_bridge:** Official ROS 2 Humble bridge for Gazebo Harmonic transport.
- **Gazebo Harmonic:** Standalone gz-sim physics and sensor simulation suite.

7. Comprehensive Workspace File Manifest

Every file in the workspace has a distinct, isolated responsibility following clean architecture principles. Perception and math routines are kept in pure Python modules without ROS dependencies, allowing full unit-test coverage without a simulator.

File Path	Package / Subsystem	Detailed Functional Responsibility (Natural Language)
src/road_survey/road_survey/dept h.py	road_survey (Perception)	Decodes 32FC1/16UC1 depth images and unprojects pixels into 3D optical points using pinhole camera matrix K with strided vectorization.
src/road_survey/road_survey/frames.py	road_survey (Geometry)	Mathematical spatial transforms connecting camera optical frame, camera physical link, UAV FRD body, PX4 NED, and ROS ENU map frames.
src/road_survey/road_survey/grid.py	road_survey (Mapping)	2.5D elevation grid ring-buffer data structure (0.25m resolution). Accumulates running min, max, mean elevation, and point counts.
src/road_survey/road_survey/risk.py	road_survey (Terrain Analysis)	Fits local planes using separable box filters; computes slope angle, step obstacle height, and roughness about the fitted plane; evaluates road shelf risk.
src/road_survey/road_survey/costmap.py	road_survey (Costmap)	Morphological cleaning of binary road masks, island rejection, distance-to-edge calculation, and conversion to ROS OccupancyGrid (0-100).
src/road_survey/road_survey/centerline.py	road_survey (Path Extraction)	Extracts medial axis skeleton of the road, traces the longest continuous topological path, prunes dead-end corner forks, and fits cubic B-splines.
src/road_survey/road_survey/terrain_mapper_node.py	road_survey (ROS Node)	ROS 2 node subscribing to /uav/depth and PX4 odometry. Publishes /road/costmap, 3D /terrain/pointcloud, TF map->uav_base_link; provides ~/save & ~/load services.
src/road_survey/road_survey/map_publisher_node.py	road_survey (Replay)	Replays previously saved road_map.npz files as latched OccupancyGrid on /road/costmap when skipping Stage 1 survey.
src/road_survey/road_survey/mapi o.py	road_survey (IO)	Serialization and deserialization utilities saving costmaps and elevations to .npz, .png, .yaml, and standard ROS .pgm map-server formats.
src/road_survey/road_survey/tune_offline.py	road_survey (CLI Tool)	Standalone offline tuning script to sweep slope, step, and roughness parameters against saved survey .npz files without flying.
src/guidance/guidance/px4.py	guidance (Autopilot Bridge)	PX4 Offboard protocol interface: manages arming, mode switching, heartbeat setpoint streaming, and versioned topic subscription.
src/guidance/guidance/explore.py	guidance (Frontier Planning)	Casts a 5m forward circular sampling ring on the costmap, clusters driveable road cells within an 80° cone, and returns the next survey waypoint.
src/guidance/guidance/aruco.py	guidance (Vision)	OpenCV ArUco detector (Dictionary 4x4_50, ID 0) with sub-pixel corner refinement and solvePnP camera-to-marker pose solver.
src/guidance/guidance/pursuit.py	guidance (Control)	Pure Pursuit path follower with adaptive lookahead (Ld=1.5m), in-place spin for heading errors >45°, curvature damping, and goal arrival logic.
src/guidance/guidance/mission.py	guidance (Logic FSM)	Pure-logic Finite State Machine: TAKEOFF -> SURVEY -> RETURN -> ACQUIRE -> TRACK -> DONE. Manages breadcrumbs, carrot setpoints, and early exits.
src/guidance/guidance/mission_node.py	guidance (ROS Node)	Interactive ROS 2 mission controller: prompts user for Survey vs Guidance (Autonomous vs Manual), handles early map save, streams PX4 setpoints.
src/guidance/guidance/localizer_node.py	guidance (ROS Node)	Subscribes to /uav/rgb, camera_info, and PX4 pose; detects roof marker and publishes /ugv/pose in map frame at 30 Hz.
src/guidance/guidance/follower_node.py	guidance (ROS Node)	Subscribes to /ugv/pose and /road/costmap; builds centreline and streams /cmd_vel steering commands to rover; publishes cross-track error.
src/guidance/guidance/pose_check_node.py	guidance (Validation)	Real-time ground-truth validator: compares ArUco /ugv/pose against Gazebo /ugv/ground_truth and evaluates centreline deviation.
src/bringup/launch/sim.launch.py	bringup (Simulation)	Launches Gazebo world, PX4 SITL daemon, MicroXRCEAgent, UGV spawner, UAV camera bridge, and static world->map TF. Configures Domain ID 42.

src/bringup/launch/mission.launch.py	bringup (Mission Launch)	Launches terrain_mapper, ugv_localizer, ugv_follower, pose_check, RViz2, and interactive mission node with emulate_tty.
src/bringup/launch/rviz.launch.py	bringup (Visualization)	Dedicated standalone launcher for RViz2 pre-configured with depth, rgb, 3D pointcloud, costmap, and vehicle poses.
src/bringup/config/world_poses.yaml	bringup (Config)	Precise spawn coordinates and headings for UAV and UGV across all competition environments (drdo_world1, world2, world3).
src/ackermann_gz_bringup/models/ackermann_bot/model.sdf	ackermann_gz_bringup (Model)	4-wheel skid-steer rover model with roof ArUco marker plate and corrected joint rotation axes (0 0 -1).
src/drdo_gz_worlds/models/x500_depth_down/model.sdf	drdo_gz_worlds (Model)	PX4 x500 quadrotor airframe equipped with downward-facing OakD-Lite depth and RGB camera sensors.

8. Standard Operating Procedures (How to Run)

The system has been completely decoupled from custom shell scripts, operating via standard two-terminal ROS 2 launch commands:

Terminal 1 – Bring Up Simulation:

```
source /opt/ros/humble/setup.bash && source ~/px4_ros_ws/install/setup.bash && source
~/uav_guided_ugv/install/setup.bash
ros2 launch bringup sim.launch.py world:=drdo_world2
Wait for PX4 to display: INFO [commander] Ready for takeoff!
```

Terminal 2 – Launch Interactive Mission & Perception:

```
source /opt/ros/humble/setup.bash && source ~/px4_ros_ws/install/setup.bash && source
~/uav_guided_ugv/install/setup.bash
ros2 launch bringup mission.launch.py
Interactive menu prompts for [1] Survey or [2] Guidance. Press [Enter] anytime to save map and
transition to UGV escort.
```

Terminal 3 (Optional) – Standalone RViz Visualization:

```
ros2 launch bringup rviz.launch.py
```